

System Design Basics

Lesson 4: Data Management & Storage

Data Management & Storage – Study Notes

Key Concepts

1. **Database Types** – Relational (SQL) vs. NoSQL (Document, Key Value, Column Family, Graph).
2. **Consistency Models** – Strong, eventual, causal, and how they trade off latency and availability.
3. **Sharding & Partitioning** – Horizontal vs. vertical sharding, key design, and how shards improve scalability.
4. **Read Heavy Analytics Patterns** – Column family stores, data warehousing, and OLAP vs. OLTP.
5. **Backup & Recovery** – Point in time recovery, snapshotting, and cross region replication.

Summary

Choosing the right storage solution starts with understanding the workload. For a **read heavy analytics** application, a column family NoSQL database (e.g., Apache Cassandra, ScyllaDB, or Amazon DynamoDB) is often preferred because it stores data in wide rows that can be scanned quickly, and it scales horizontally by adding nodes. Consistency can be tuned per query: for dashboards you might accept eventual consistency to keep latency low, while critical metrics may require strong consistency.

Sharding is the key to scaling. Horizontal sharding distributes rows across many nodes based on a hash or range of a partition key. Proper key design prevents hotspots and ensures even data distribution. Backup strategies such as incremental snapshots and cross region replication protect against data loss without impacting read performance.

Important Points

- **SQL vs. NoSQL**
 - SQL: ACID, fixed schema, joins.
 - NoSQL: flexible schema, horizontal scaling, eventual consistency (unless configured otherwise).
- **Consistency Trade offs**
 - **Strong consistency**: single source of truth, higher latency.
 - **Eventual consistency**: low latency, possible stale reads.
 - **Causal consistency**: guarantees order of causally related operations.
- **Sharding Strategies**
 - **Hash sharding**: uniform distribution, but hard to range query.
 - **Range sharding**: efficient for range queries, but can lead to hotspots.
 - **Consistent hashing**: smooth rebalancing when nodes join/leave.
- **Read Heavy Analytics**
 - Use **wide columns** or **column families** to store time series data.
 - Employ **materialized views** or **pre aggregated tables** for fast reporting.

- Consider **data warehouses** (Snowflake, BigQuery) for complex analytics.
 - ****Backup & Recovery****
- **Snapshotting**: periodic full or incremental copies.
- **Cross region replication**: protects against site wide failures.
- **Point in time recovery**: essential for compliance and audit.

Examples

1. Choosing a Database for a Real Time Analytics Dashboard

| Requirement | SQL (PostgreSQL) | NoSQL (Cassandra) |

|-----|-----|-----|

| Read latency | 10–50 /ms (indexing) | 1–5 /ms (wide rows) |

| Write throughput | Limited by single node | Unlimited (add nodes) |

| Schema evolution | Rigid | Flexible |

| Consistency | Strong (default) | Tunable (e.g., QUORUM) |

****Decision****: For a dashboard that aggregates millions of events per second, Cassandra's low latency reads and horizontal scalability make it the better fit.

2. Sharding a User Profile Service

- ****Partition key****: `user\_id` (hash sharded).
- ****Secondary index****: `country` (for regional analytics).
- ****Shard count****: Start with 8 nodes; use consistent hashing so adding a node moves only ~12.5% of data.
- ****Backup****: Daily snapshots to S3, incremental nightly syncs, and cross region replication to a standby cluster.

Quick Review

1. What are the main differences between relational and NoSQL databases?
2. Explain the trade off between strong and eventual consistency.
3. Why is horizontal sharding preferred for read heavy workloads?
4. What is a column family store, and how does it benefit analytics?
5. Describe two backup strategies that protect against data loss without impacting read performance.

Further Reading

- ****CAP Theorem**** – Understanding Consistency, Availability, Partition Tolerance.
- ****Consistent Hashing**** – Detailed algorithms and use cases.
- ****Data Warehousing vs. OLAP**** – Architectural differences and tooling.
- ****Backup & Disaster Recovery**** – Best practices for cloud native databases.
- ****Time Series Databases**** – InfluxDB, TimescaleDB for specialized analytics workloads.

